

EXERCÍCIO 1: Classes em JavaScript

Objetivo Geral

Crie uma estrutura de herança onde as classes **Gato**, **Cachorro** e **Pato** herdam da classe base **Animal**.

1. Classe Animal (Base)

- **Atributos:** nome e tutor.
- **Métodos:** EmitirSom() e Comer().
- **Encapsulamento:** Implemente métodos *Gets* e *Sets* para cada atributo.

2. Implementação e Teste

- **Polimorfismo:** Cada animal deve emitir seu som específico: “Au au”, “Miau” ou “Quack”.
- **Instanciação:** Crie objetos de teste para validar o funcionamento das classes.



EXERCÍCIO 2

- Usando 2 <input>, receber preço e quantidade e calcular o valor total (preço * quantidade) e exibir o valor total abaixo.
- Quando um deles for alterado, alterar automaticamente o valor total.

EXERCÍCIO 3

- Crie uma página web que mostra uma lista de alunos usando `<ul>` e `<li>`. Após criar pelo menos 6 nomes de alunos, crie o código CSS e Javascript para alterar a cor do texto para vermelhos dos alunos que possuem uma posição par na lista.

Dica: use o `querySelectorAll` e o `for`

Lista de Alunos

João

Maria

Pedro

Ana

Lucas

Exercício 4: Operador Spread e Arrow Functions

Você está desenvolvendo a tela de "Finalizar Compra" de uma loja virtual. O sistema recebe os dados do usuário, de faturamento e duas listas de produtos (estoque local e parceiros).

Sua tarefa é:

- Unir `estoqueLocal` e `estoqueParceiro` em `todosProdutos` usando Spread.
- Criar `resumoPedido` unindo `cliente` e `entrega` (o CEP de "entrega" deve prevalecer).
- Criar Arrow Function `gerarRecibo` que retorne uma string formatada com Template Literals.

Dados iniciais:

```
const cliente = { nome: "Ana Silva", cpf: "111.222.333-44", cep: "00000-000" };

const entrega = { frete: 15.00, cep: "63900-000" };

const estoqueLocal = ["Notebook", "Mouse"];

const estoqueParceiro = ["Teclado Mecânico", "Monitor"];
```

Formato do Recibo: "O cliente [NOME] receberá [QUANTIDADE DE PRODUTOS] itens no CEP [CEP]"

EXERCÍCIO 5: POO E ENCAPSULAMENTO

Uma startup financeira está criando seu novo sistema de contas e você ficou responsável por implementar a classe `ContaCorrente`. O saldo do cliente não pode ser acessado de forma direta para evitar que outros programadores alterem o valor de forma arbitrária.

Sua tarefa é:

01 Criar uma **classe moderna** chamada `ContaCorrente` usando a palavra-chave `class` e o método `constructor` para titular e `saldoInicial`.

02 Declarar o saldo como uma **variável privada** (utilizando a sintaxe `#`) para impedir acesso externo direto.

03 Implementar um método `get saldo()` que retorne uma string formatada: `"R$ [VALOR]"`, utilizando o método `.toFixed(2)`.

04 Implementar um método `set saldo(novoValor)` que valide se o valor é **positivo**. Caso contrário, exiba um erro no console.

EXERCÍCIO 6: DOM e JS

Você é o desenvolvedor front-end de uma loja e recebeu a tarefa de ativar o **"Modo Black Friday"** (crie um botão para isso) na página principal via JavaScript. O HTML da sua página possui a seguinte estrutura básica:

```
<!-- Estrutura HTML de base -->

<div class="produto-card">Smartphone</div>
<div class="produto-card">Notebook</div>
<div class="produto-card">Fone Bluetooth</div>
<p id="cupom-secreto" style="display: none">CUPOM: JSBLACK20</p>
```

Sua tarefa é escrever o código JavaScript que execute os seguintes passos:

1. Troca do Banner (Manipulação de Atributos)

Capture a imagem do banner usando `document.getElementById()`. Utilize `.setAttribute()` para alterar o `src` para `"banner-black-friday.jpg"`.

2. Destaque nos Produtos (Manipulação de Classes)

Use `document.querySelectorAll()` para capturar a classe `produto-card`. Itere sobre a lista (*for* ou *forEach*) e aplique `.classList.add('black-friday-theme')`.

3. Resgate do Cupom (Texto Oculto)

Capture o elemento `#cupom-secreto`. Considerando que está com `display: none`, utilize a propriedade correta (`.innerText` vs `.textContent`) para extrair o texto e exibi-lo no console.